



**DEBRE BERHAN UNIVERSITY**

**College of Computing**

## Chapter 2: Programming with Event Driven in C#



By Worku Muluye

August 13, 2026

# Outline

- C# Fundamentals:
  - Data types,
  - Variables,
  - Operators,
  - Control Flow Statements,
  - Arrays,
  - Strings,
  - Functions...
- OOP
- Event
- Delegate
- Namespace and Library



# Introduction to C# Programming Language

- C# (C-Sharp): A modern, object-oriented, general-purpose programming language developed by Microsoft by Anders Hejlsberg in 2002.
- Runs on .NET Framework, .NET Core, and .NET 5/6+ platforms.
- Approved by ECMA and ISO standards bodies.
- Designed to combine the power of C++, the simplicity of Java, and the productivity of Visual Basic.
- Introduced to overcome limitations of C and C++.
- Aimed to simplify programming, support web and internet-based applications, and achieve platform independence.

# Why C#?

- **Simple and familiar:** Resembles C/C++, hides complex feature.
- **Portable:** Supports both source code and IL code portability.
- **Architecturally Neutral:** Consistent behavior across systems due to **Common Type System**.
- **Secure:** Runs under Common Language Runtime, providing a protected execution environment.
- **Robust:** Strong type-checking prevents common programming errors.
- **Distributed:** Ideal for building multi-client/shared-resource systems (e.g., WCF, RESTful APIs).
- **Multithreaded:** Supports concurrent task execution using multiple threads.

# Why C#?

- **Dynamic:** Supports runtime type resolution (dynamic keyword).
- **Compiled & Interpreted:** Uses a two-stage process (CSC -> MSIL -> JIT compilation).
- **Object-Oriented:** Fully supports Encapsulation, Inheritance, Polymorphism, and Abstraction.
- **Platform Independent:** With .NET Core/.NET, runs on Windows, Linux, and macOS.
- **Automatic Memory Management:** Garbage Collector handles allocation and deallocation.
- **Exception Handling:** Strong built-in mechanisms to handle runtime errors gracefully.

# Data Types

- A data type specifies the size and type of variable values.
- Use the correct data type for the corresponding variable to:
  - Avoid errors,
  - Save time and memory,
  - Make your code more maintainable and readable. The most common data types are:

| Data Type | Size                  | Description                                                                       |
|-----------|-----------------------|-----------------------------------------------------------------------------------|
| int       | 4 bytes               | Stores whole numbers from -2,147,483,648 to 2,147,483,647                         |
| long      | 8 bytes               | Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 |
| float     | 4 bytes               | Stores fractional numbers & sufficient for storing 6 to 7 decimal digits          |
| double    | 8 bytes               | Stores fractional numbers & sufficient for storing 15 decimal digits              |
| bool      | 1 byte                | Stores true or false values                                                       |
| char      | 2 bytes               | Stores a single character/letter, surrounded by single quotes                     |
| string    | 2 bytes per character | Stores a sequence of characters, surrounded by double quotes                      |

# Syntax

- We created a C# file named Program.cs with the following code to print "Hello World" to the screen:
- Every statement ends with semicolon (;)
- C# is case-sensitive (MyClass ≠ myclass)
- Single-line comments // and multi-line comments /\* \*/.
- **using System** allows access to classes in the System namespace.
- **namespace** groups related classes and organizes code.
- **Curly braces {}** define the start and end of a code block.
- **class** defines a container for data & methods. All code runs inside a class named *Program*.
- **Main** method is the entry point of a C# program. Code inside {} runs when the program starts.
- **Console.WriteLine()** prints text. Console comes from the System namespace.

```
//Program.cs
using System;

namespace HelloWorld
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

# Variables

- Variables are containers for storing data values. It is a named storage location in a memory that holds data which can be changed during program execution.
- **Rules:**
  - Every variable must have a data type before its name. E.g., int, string, bool, float, double...
  - Every variable must have a name (identifier).
  - A variable name must be unique within the same scope.
  - A variable must be declared before it is used.
  - A variable must be assigned a value before it is used.
  - Variable name is case sensitive(C# is case sensitive).
  - Naming rules:
    - Variable name must begin with a letter or underscore(\_).
    - Cannot start with a number.
    - Cannot contain space or special characters except \_.
    - Cannot be a C# keyword like int, void, class, public ...
- Variables are only accessible within the block{} where they are declared.



# Variables: Types of Variables

## ■ Local Variables

- Declared inside a method, constructor, or block { }.
- Exist only while the method is running.
- Must be initialized before use.

```
void Var1 ()  
{  
    int num = 20; // local variable  
    Console.WriteLine(num);  
}
```

## ■ Instance Variables (Non-Static Fields)

- Declared inside a class but outside methods.
- Each object of the class gets its own copy.

```
class Person  
{  
    string name; // instance variable  
    int age; // instance variable  
}
```

## ■ Static Variables

- Declared with the **static** keyword.
- Belongs to the class, not to objects.
- Shared among all objects of the class.

```
class Counter  
{ // static variable  
    public static int count = 0;  
}
```

# Variables: Types of Variables

## ■ Constant Variables

- Declared with the const keyword.
- Value cannot change after initialization.
- Must be assigned at compile time.

```
const double PI = 3.14159;
```

## ■ Read-Only Variables

- Declared with the readonly keyword.
- Value can be assigned only once, at declaration or inside a constructor.

```
readonly int std;  
public staff(int newId)  
{  
    std = newId; // allowed in constructor  
}
```

## ■ Global Variables (via static)

- C# doesn't allow direct global variables.
- But you can simulate them using static variables in a class.

```
class Counter  
{ // static variable  
    public static int count = 0;  
}
```

## ■ Parameters and Arguments

- Method Parameters: variables passed to methods.

```
void PrintName(string name) //  
parameter  
{  
    Console.WriteLine(name); }
```

# Variables: Types of Variables

## ■ Types of variables in memory management:

### ■ Value Types

- Stored directly in stack memory.
- Hold the actual data.
- When assigned to another variable, a copy of the value is made.
- Changing one variable does not affect the other.

```
int x = 10;  
int y = x; // copy of x  
y = 20;  
Console.WriteLine(x); // 10 (unchanged)
```

### ■ Reference Types

- Store a reference (address/pointer) in the stack, but the actual data lives in the heap.
- When assigned, both variables point to the same object in the heap.
- Changing one affects the other.

### ■ Pointers (Unsafe in C#)

- Pointers hold the memory address of a variable.
- They are not commonly used in C#
- But possible with the unsafe keyword.

```
class Staff { // reference type  
    public string Name;  
}  
Staff s1 = new Staff();  
s1.Name = "Kebron";  
Staff st2 = s1; //st2 points to same object as st1  
st2.Name = "Simon";  
Console.WriteLine(st1.Name); // Simon changed
```

# Variables

- Syntax: datatype variablename;
- For example:
  - `int age = 23;` stores integers (whole numbers), without decimals.
  - `double tax = 19.9;` stores floating point numbers, with decimals
  - `char grade = 'A';` stores single characters.
  - `string name = "Dawit";` stores text.
  - `bool isMale = true;` stores values with two states: true or false
- **Declare multiple variables:** To declare more than one variable of the same type, use a comma-separated list. E.g. `int x = 5, y = 6, z = 50;`

# Output and User Input

## ■ Output:

- **WriteLine() Method:** Output values/print text with adding a new line. `Console.WriteLine("Hello World");`
- **Write() Method:** Print output values without adding a new line. `Console.Write("Hello World ");`
- We can add as many `WriteLine()` or `Write()` methods as you want.

## ■ User Input:

- **ReadLine() Method:** read user input as a **string**. Example:
  - `Console.WriteLine("Enter username:");` // ask for username
  - `string userName = Console.ReadLine();` // read user input
  - `Console.WriteLine("Username is: " + userName);` // display username
- **Note:** `Console.ReadLine()` returns a string, so to store numbers, you must convert the input.
  - `int age = Console.ReadLine();` // error: cannot convert string to int
  - `Console.WriteLine("Enter your age:");`
  - `int age = Convert.ToInt32(Console.ReadLine());`
  - `Console.WriteLine("Your age is: " + age);`

# Operators

- Operators are used to perform operations on variables and values.
- **Arithmetic Operators:** used to perform common mathematical operations.
  - `+`, `-`, `*`, `/`: Adds, subtracts, multiplies, and divides values respectively.
  - `%` Modulus: Returns the division remainder.
  - `++` Increment: Increases the value of a variable by 1, and
  - `--` Decrement: decreases the value of a variable by 1.
- **Assignment Operators:** used to assign values to variables. E.g., `int x = 10;`
  - The addition assignment operator (`+=`) adds a value to a variable.
  - Example, `int x = 10; x += 5;`

# Operators

- **Comparison Operators:** used to compare two values (or variables).
  - `==`, `!=`, `>`, `<`, `>=`, `<=`: are the symbols for equal to, not equal to, greater than, less than, greater than or equal to, and less than or equal to, respectively.
- **Logical Operators:** used to determine the logic between variables or values.
  - `&&`, `||`, `!`: are the symbols for logical and (returns true if both conditions are true), logical or (returns true if at least one condition is true), and logical not (reverses the result), respectively.

# Control Flow Statements in C#

- Control Flow Statements **alter** the **normal sequence of program execution**.
- By default, statements are executed sequentially (from top to bottom).
- Control flow enables decision-making, looping, and branching.
- **Purpose of Control Flow Statements:**
  - Provide better control over execution.
  - Allow skipping, repeating, or jumping between statements.
  - Essential for writing dynamic, flexible, and complex programs.



# Control Flow Statements in C#

## ■ Conditionals/Decision-Making Statements

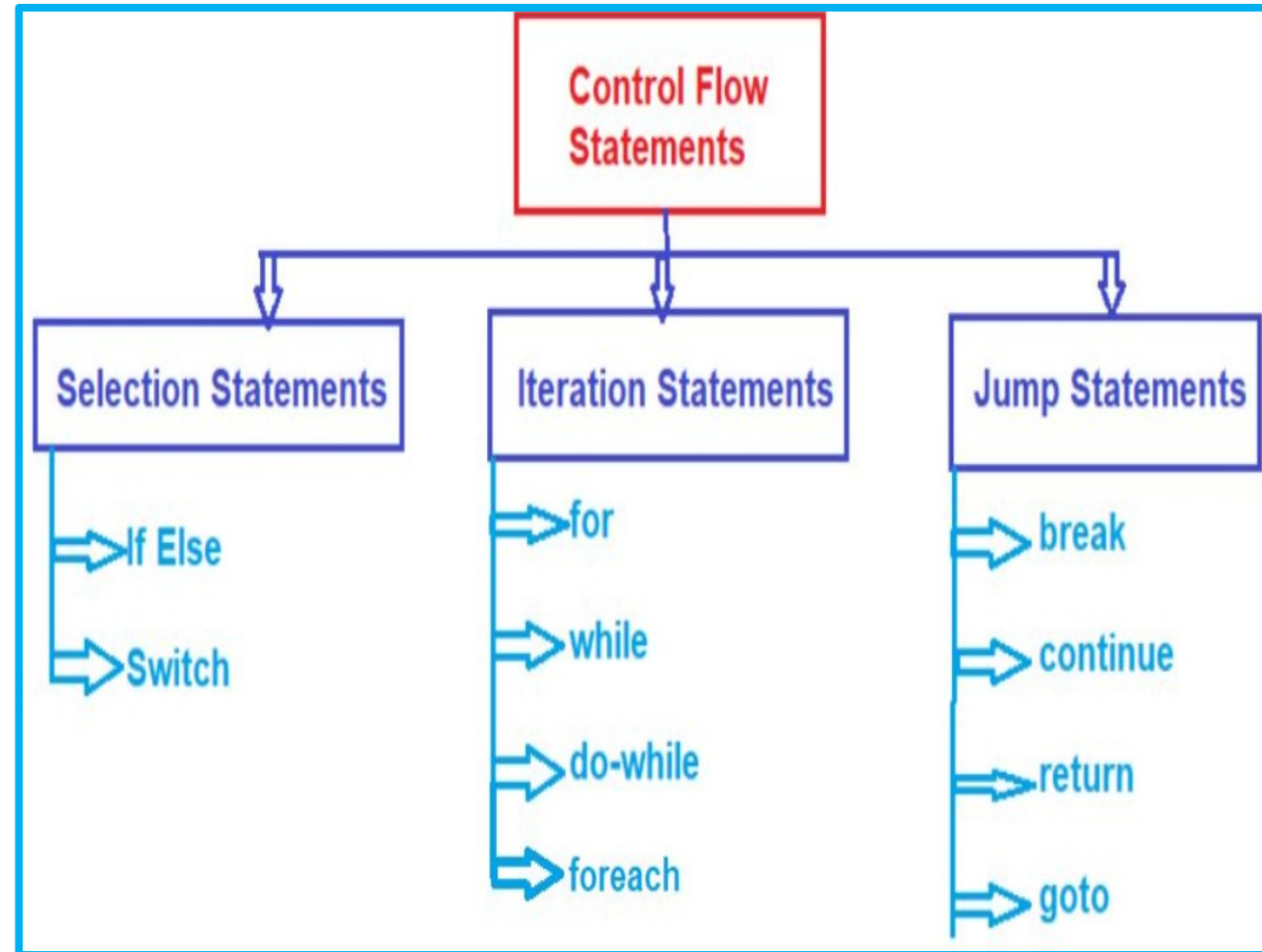
- Used to execute blocks of code based on conditions.
- Examples: if, if-else, if-else if, switch.

## ■ Looping

- Used for executing statements repeatedly.
- Examples: for, while, do-while, foreach.

## ■ Branching/Jump

- Used to alter execution flow directly.
- Examples: break, continue, goto, return, throw.



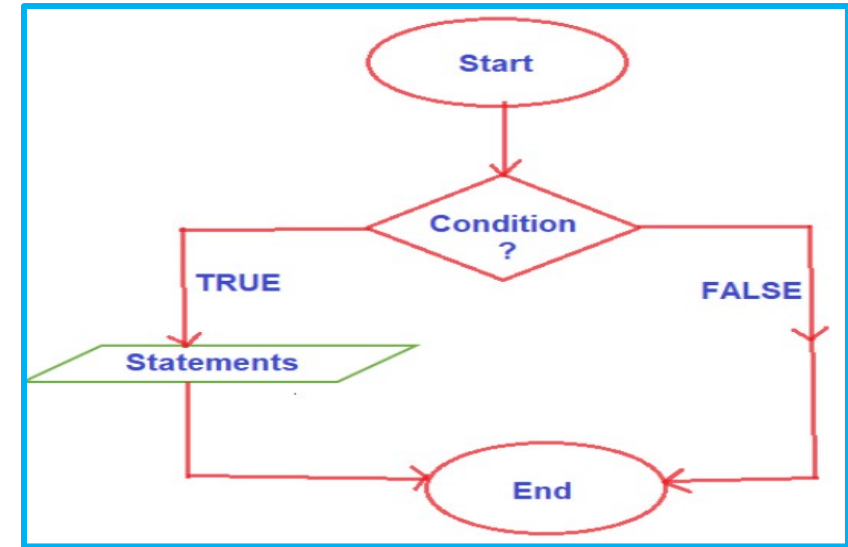
# Control Flow Statements: If-Else Statements

- If-Else Statements Also called Decision-Making Statements.
- Allow execution of different code blocks based on conditions.

## ■ If Statement

- Executes a block of code only if a condition is true.

```
if(condition)
{
    Statements;
}
```



## ■ If Without Braces

- If braces {} are omitted, only the first statement belongs to the if.

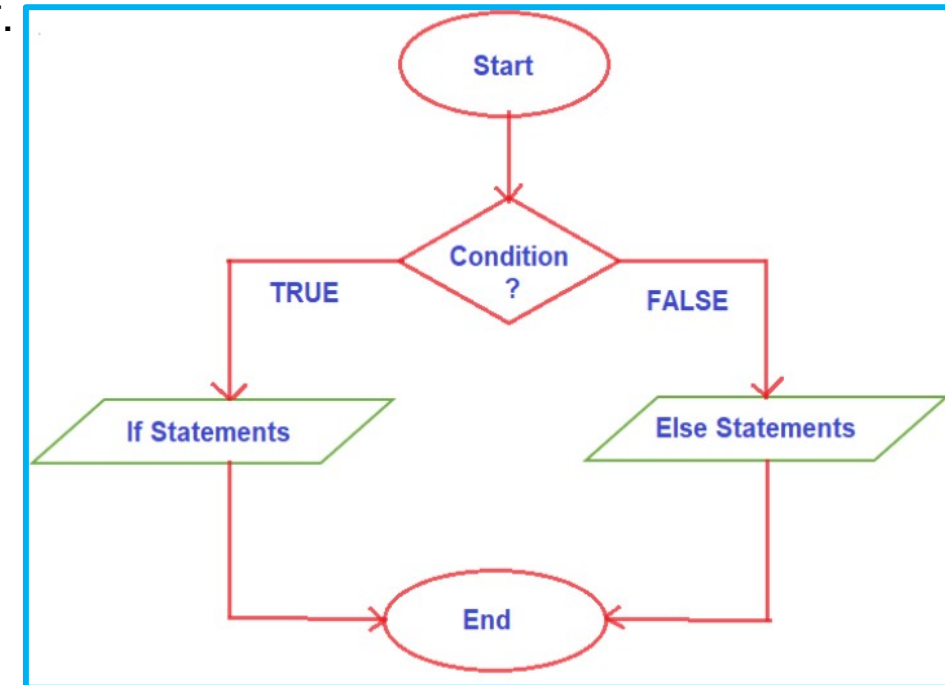
```
if (condition)
statement;
```

## ■ If-Else Statement

- Executes one block if the condition is true, another block if false.
- Only one block executes at a time.

```
If (Condition)
{
    if statements;
}
else
{
    else statements;
}
```

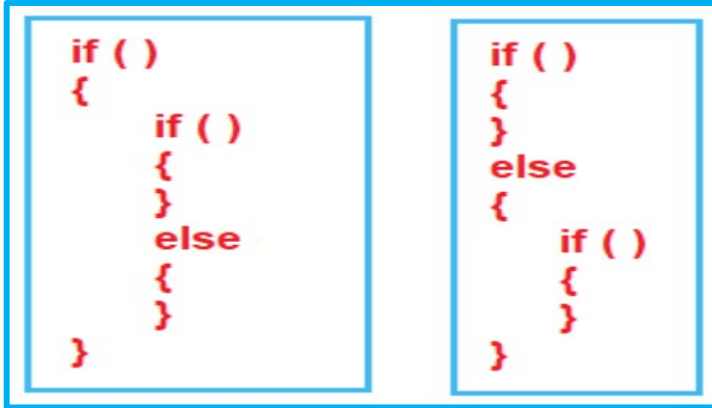
**Else:** Executes code if the condition in the if is false.



# Control Flow Statements: If-Else Statements

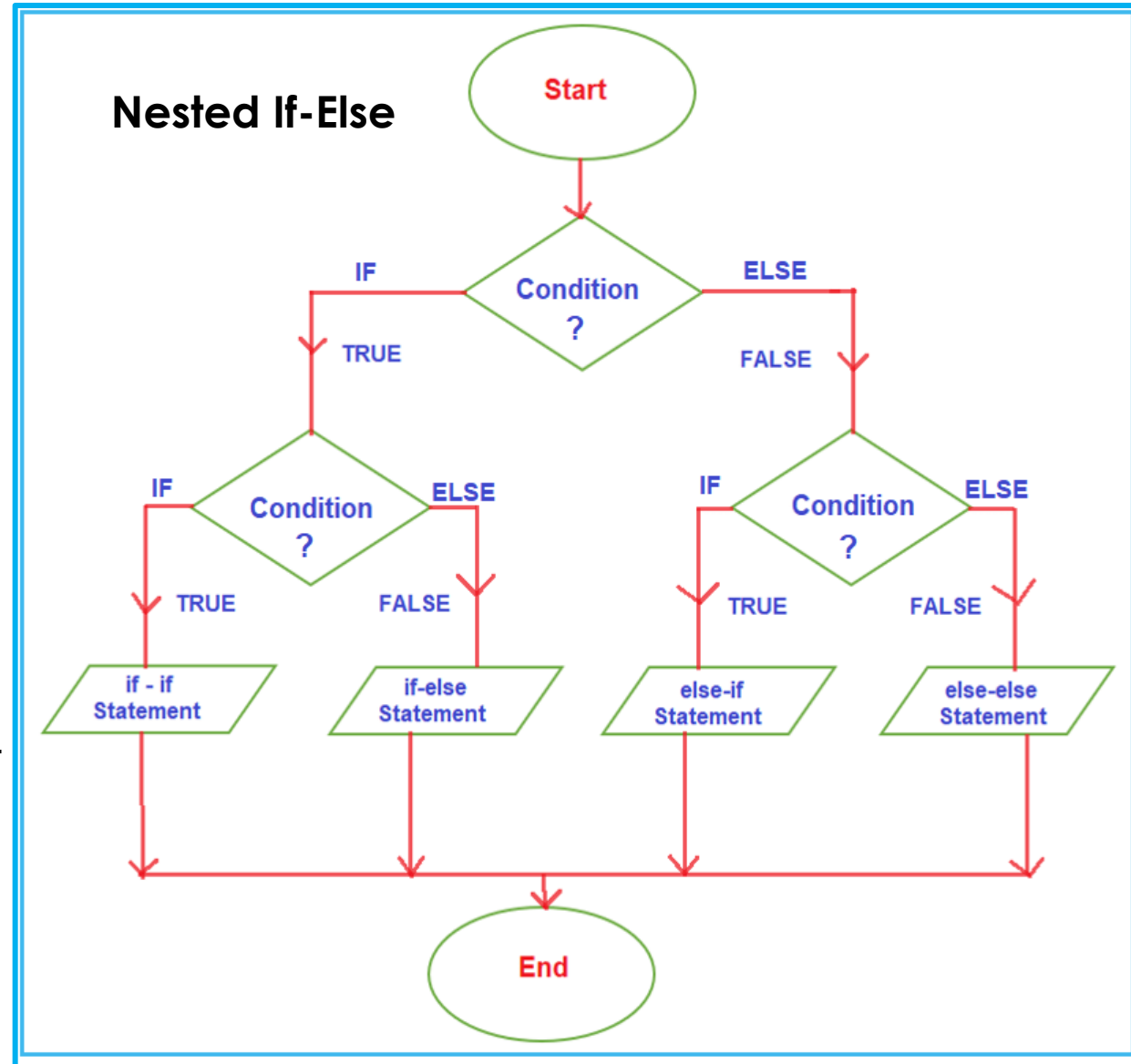
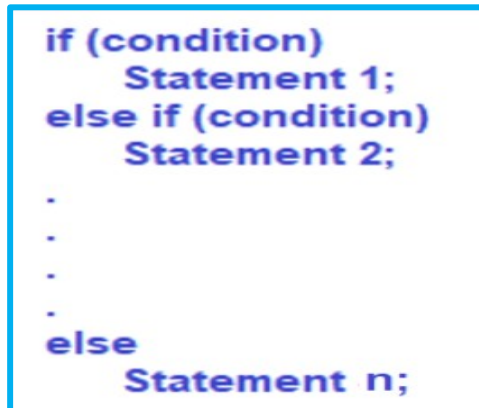
## ■ Nested If-Else

- An if-else inside another if or else.
- Useful when decisions depend on multiple conditions.



## ■ If-Else-If Ladder

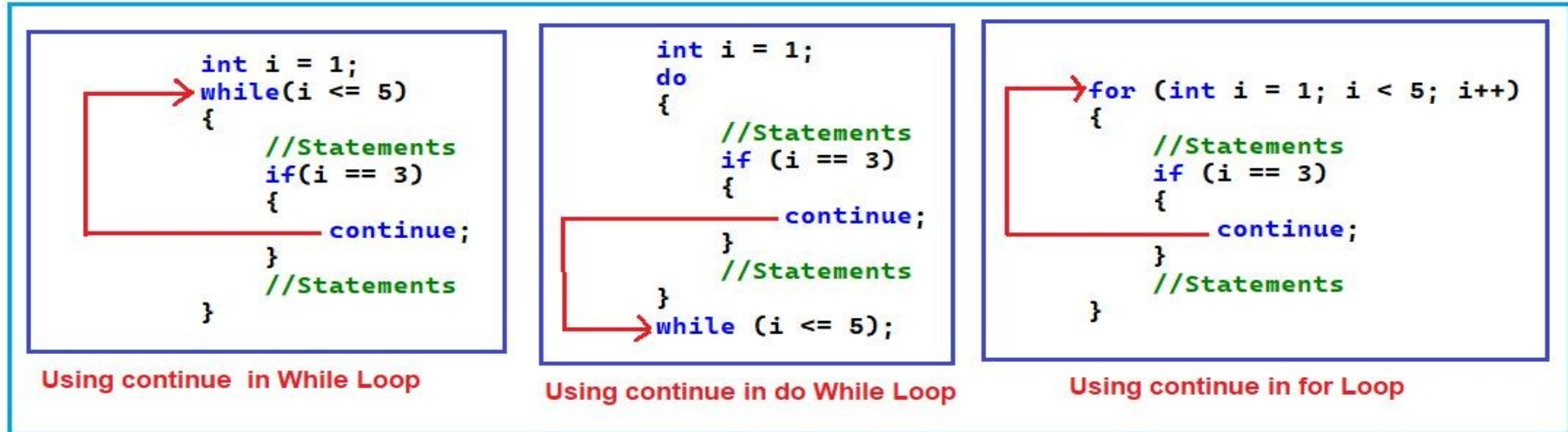
- Multiple conditions are checked top to bottom.
- First true condition's block executes; others are skipped.



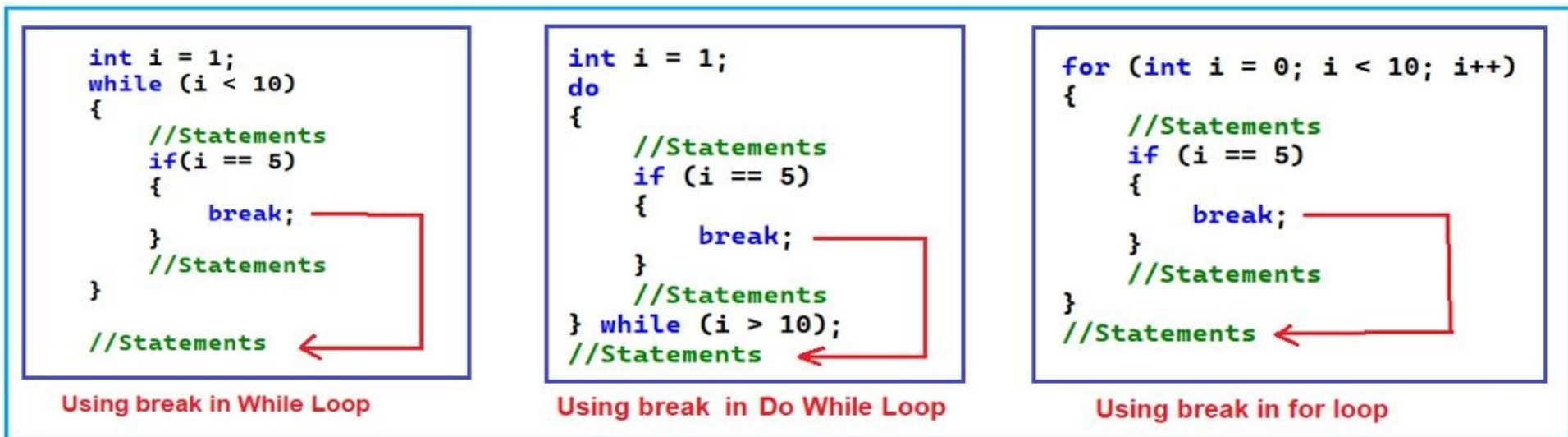
**else if:** Checks another condition if the previous if was false.

# Control Flow Statements: Continue & Break Statements

- **Continue Statement:** is a jump statement used inside loops. It skips the remaining statements of the current iteration & moves control to the next loop iteration.



- **Break Statement:** A keyword that terminates a loop (`for`, `while`, `do-while`, `foreach`) and `switch` statement



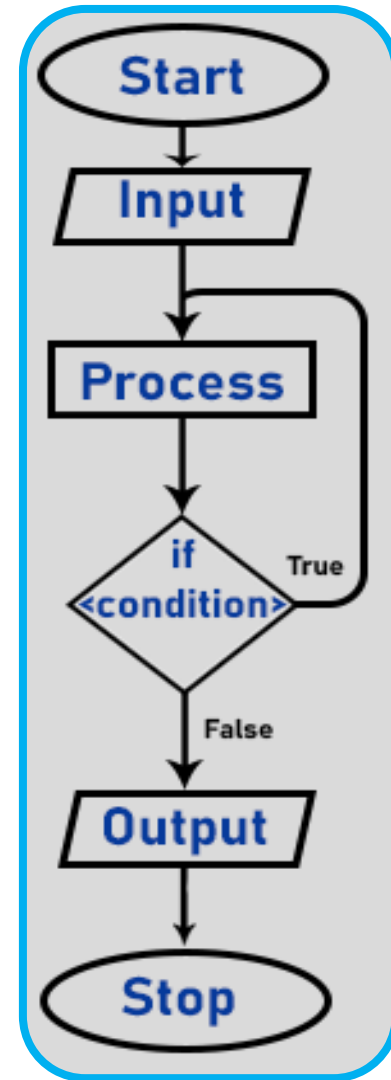
# Control Flow Statements: Switch Statements

- A **switch statement** is a multi-way branching statement.
- It evaluates an expression and transfers execution to the matching case label.
- It is an alternative to multiple if-else-if statements, making code more readable and organized.
- **Rules:**
  - **Expressions** can be of int, char, string, enum, or nullable types.
  - **case** values must be unique constants.
  - **break** ends the case; without it, control “falls through”.
  - **default** is optional; executes if no match is found.
- Why use a Switch instead of an If-Else?
  - If-Else: better for complex boolean conditions.
  - Switch: cleaner for multiple fixed values.
  - Improves readability, maintainability, and efficiency.

```
switch(variable) //expression
{
    case 1:
        //execute your code
        break;
    case n:
        //execute your code
        break;
    default:
        //execute your code
        break;
}
```

# Loops

- Loops are used to execute a block of code repeatedly as long as a **condition** is true.
- Types of Loops:**
  - for:** Runs a block of code a specific number of times.
  - while:** loops through a block of code as long as a specified condition is true.
  - do/while:** loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.
  - foreach:** Loops through elements in a collection or array.
- Note,** do not forget to increase the variable used in the condition, otherwise the loop will never end.



# Loops: While Loop

- **While Loop:** loops through a block of code as long as a specified condition is true.
  - Condition is checked.
  - If true, body executes.
  - After execution, the condition is checked again.
  - Repeats until the condition becomes false.
- It is an entry-controlled loop (condition checked before execution).
- **Nested While Loop** (While Loop inside another Loop):

## Syntax:

```
while (Condition)
{
    Statements;
}
```

```
int i = 0;
while (i < 5)
{
    Console.WriteLine(i);
    i++;
}
```

## Syntax:

```
while (outer condition)
{
    Outer while Statements;

    while (inner condition)
    {
        Inner while Statements;
    }

    Outer while Statements;
}
```



## Loops: Do/While Loop

- **Do/While:** loop executes the code block **once, before checking** if the condition is true, then it will repeat the loop as long as the condition is true.
  - It is a post-tested or exit-controlled loop.
  - Statements inside the loop are executed at least once, even if the condition is false.
  - Commonly used in menu-driven programs, where the user decides when to terminate the loop.
  - Note: Condition ends with a semicolon (;)
    - Execute loop body.
    - Check the condition.
    - If true, repeat the loop body.
    - If false, exit the loop.
- **Nested Do-While Loop:** Using a do-while loop inside another do-while loop.

```
do
{
    statements;
}
while(condition);
```

```
int i = 0;
do
{
    Console.WriteLine(i);
    i++;
}
while (i < 5);
```

```
do
{
    statement n;
    do
    {
        statement n;
    }
    while(test condition);
}
while(test expression);
```



# Loops: For Loop

- **For:** Runs a block of code a specific number of times.
  - Use a for loop when you know the number of iterations is known in advance.
  - It is also called a counter-controlled loop.
  - Commonly used when repetition is based on counting rather than a condition alone.
    - Execute initialization.
    - Check the condition.
    - If true, execute the loop body.
    - Execute increment/decrement.
    - Repeat from step 2 until the condition is false.
- **Nested For Loop:** A for loop inside another for loop. Inner loop executes completely for each iteration of the outer loop.

```
for (initialization; condition; increment/decrement)
{
    // c# instructions
}
```

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
```

```
for (initialize counter; test condition; ++ or --)
{
    for (initialize counter; test condition; ++ or --)
    {
        // inner for loop
    }
    // outer for loop
}
```

## Loops: Foreach Loop

- **Foreach**: used to iterate over all elements in a **collection or array**.
- Unlike for or while loops, you do not need to manage the counter manually.
- It is read-only, meaning you cannot modify the elements of the collection directly inside the loop.

```
foreach (datatype variable in collection/array)
{
    // statements to execute
}
```

- **datatype variable**: represents each element in the collection.
- **collection**: array, list, or any enumerable collection.
- Loop executes once for each element in the collection or array.
- Take the first element of the collection.
- Assign it to the loop variable.
- Execute the loop body using the loop variable.
- Move to the next element in the collection.
- Repeat until all elements are processed.

```
string[] fruits = { "Banana", "Mango", "Orange" };
foreach (string fruit in fruits)
{
    Console.WriteLine(fruit);
}
```

# Arrays

- Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.
- To declare an array, define the variable type with square brackets.
  - `string[] cars = {"BMW", "Ford", "Toyota"};`
- **Access the Elements of an Array:** we can access an array element by referring to the index number.
  - `string[] cars = {"BMW", "Ford", "Toyota"};`
  - `Console.WriteLine(cars[0]);`
- **Change an Array Element:** To change the value of a specific element, refer to the index number.
  - `string[] cars = {"BMW", "Ford", "Toyota"};`
  - `cars[0] = "V8";`
  - `Console.WriteLine(cars[0]);`

# Arrays

- **Array Length:** To find out how many elements an array has, use the Length property.
- **Loop Through Arrays:**
  - **for loop:** Use a for loop with the array's length to iterate over elements.

```
string[] cars = { "BMW", "Ford", "Toyota"};  
for (int i = 0; i < cars.Length; i++) {  
    Console.WriteLine(cars[i]);  
}
```
  - **foreach Loop:** foreach is a simpler way to loop through arrays, without needing an **index**.

```
foreach (string car in cars){  
    Console.WriteLine(car);  
}
```
- **Note:** foreach is more readable and **avoids manual indexing**, making it **ideal** for looping through arrays.

# Arrays

- **Sort Arrays:** Use the `Array.Sort()` method to sort elements alphabetically or in ascending order.

```
string[] cars = {"BMW", "Ford", "Toyota"};
Array.Sort(cars);
foreach (string i in cars)
{
    Console.WriteLine(i);
}
```

- **Multidimensional Arrays:** an array of arrays, used to store data in table-like (row and column) format such as 2-dimensional, 3-dimensional, or multidimensional arrays.

```
int[,] numbers = { {1, 4, 2}, {3, 6, 8} };
foreach (int i in numbers)
{
    Console.WriteLine(i);
}
```

# Strings

- A string in C# is an instance of the **System.String** class representing text or whose value is text.
- Internally, it's an immutable sequence of Unicode characters.
- string is immutable, the contents cannot be altered after creation.
- Operations that appear to modify a string actually return a new string object.
- Internally, it's a sequential, read-only collection of Char objects (UTF-16 code units).
- **string** is a keyword alias for System.String; both are equivalent.
- Conventionally, use string; it works without using System;.
- The String class offers a rich set of methods for manipulation and comparison.
- You can declare strings in various ways:
  - string message1; // uninitialized
  - string message2 = "Hello World"; //initialized
  - string nullString = null; // explicitly nullable
  - System.String fullName = "Dawit"; //using System.String
  - string message3 = String.Empty; //zero-length safe initialization

# Strings

## ■ String Common Operations & Methods

- **Concatenation:** + operator, String.Concat()
- **Interpolation:** \$"Name: {name}" (Modern, readable)
- **Searching:** .Contains(), .StartsWith(), .EndsWith(), .IndexOf()
- **Modification (Returns New String):** .Replace(), .ToUpper(), .ToLower(), .Substring(), .Trim()
- **Info:** .Length (property)

## ■ String Methods:

- **Length:** Gets the number of characters
- **ToUpper():** Converts to uppercase
- **ToLower():** Converts to lowercase
- **Contains("Hello"):** Checks if substring exists
- **IndexOf("World"):** Finds index of substring
- **Replace("World", "C#"):** Replaces text
- **Substring(0, 5):** Extracts part of the string
- **Trim():** Removes whitespace from both ends
- **Concat():** combines two or more strings into a single new string.
- **StartsWith()/EndsWith():** Returns true if the string begins/ends with the specified substring.

# Functions

- A function (method) is a block of code that performs a specific task.
- Functions improve code reusability, readability, and maintainability.
- In C#, functions are usually called methods because they belong to a class.
- **In C#, a function, also called a method, typically has:**
  - **Function Name:** Identifies the method(Mandatory).
  - **Parameter List:** Defines inputs (0 or more) (Optional).
  - **Return Type:** Defines the type of value returned (Mandatory). Use void if no value is returned.
  - **Access Specifier:** Controls accessibility (public, private, protected) (Optional).
  - **Modifier:** Defines behavior (static, virtual, sealed, partial) (Optional).
  - **Function Body:** Contains the code to execute, enclosed in { }.

```
[access_modifier] [return_type] [function_name]([parameters])  
{  
    // Function body  
}
```



# Functions

## ■ Function Signature

- The combination of Function Name and Parameter List.
- Note: Return type is not part of the signature.

## ■ Return Statement

- Terminates the function execution.
- Returns a value to the calling function (if required).
- Control returns to the point immediately after the function call.

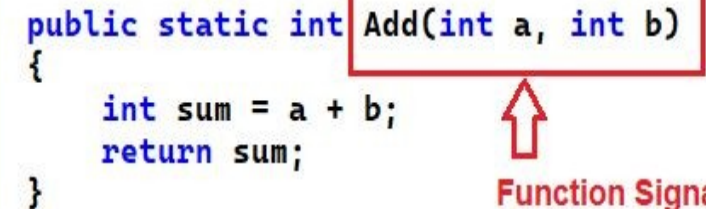
## ■ Calling a Function

- Control transfers to the called function.
- Returned value (if any) is stored in a variable or used directly.

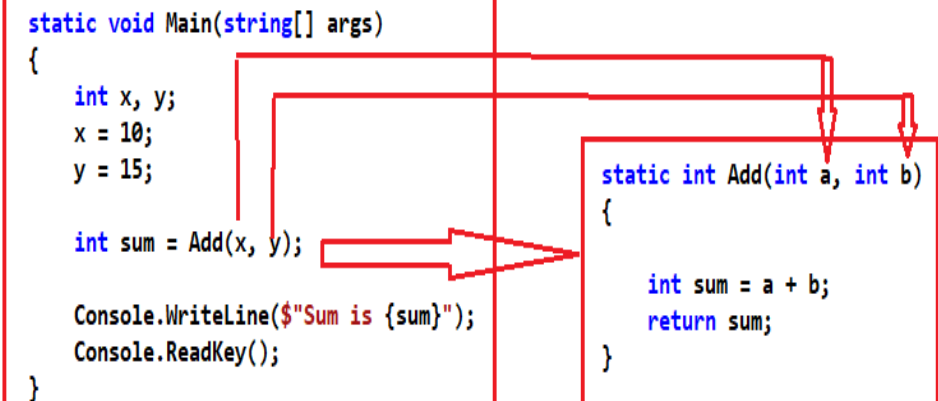
## ■ Function completes when:

- return statement is executed.
- Closing brace } of the function is reached.
- An unhandled exception occurs.

```
public static int Add(int a, int b)
{
    int sum = a + b;
    return sum;
}
```



Function Signature



```
static void Main(string[] args)
{
    int x, y;
    x = 10;
    y = 15;

    int sum = Add(x, y);

    Console.WriteLine($"Sum is {sum}");
    Console.ReadKey();
}

static int Add(int a, int b)
{
    int sum = a + b;
    return sum;
}
```

# Functions: Types of Functions

- **Built-in Functions:** Functions already defined by .NET / .NET framework.
  - It is already tested and optimized, saves development time, and is reliable and widely used.
  - It cannot customize beyond the function's original purpose.
  - Examples: `Console.WriteLine()`, `Math.Sqrt()`, `String.Length()`.

# Functions: Types of Functions

- **User-defined Functions:** Created by the programmer to perform specific tasks.
  - Created by the programmer to perform specific tasks.
  - Reduces code duplication, improves readability, and optimizes program size.
  - Fully customizable according to project requirements.

```
public void Greet() {  
    Console.WriteLine("Welcome!");  
}
```

```
class Program  
{  
    static int Add(int a, int b)  
    {  
        int sum = a + b;  
        return sum;  
    }  
  
    static void Main(string[] args)  
    {  
        int x, y;  
        x = 10;  
        y = 15;  
  
        int sum = Add(x, y);  
  
        Console.WriteLine($"Sum is {sum}");  
        Console.ReadKey();  
    }  
}
```

The diagram illustrates the relationship between formal and actual parameters. In the `Add` function, `int a` and `int b` are formal parameters. In the `Main` method, `x` and `y` are actual parameters. Red arrows show the flow of data from the actual parameters in `Main` to the formal parameters in `Add`.

# Functions: Types of Functions

## ■ Functions with Return Type:

- Return a value to the caller.

```
public string GetName() {  
    return "Degu"; }
```

## ■ Parameterized Functions:

- Functions that take inputs.

```
public int Add(int a, int b) {  
    return a + b; }
```

## ■ Void Functions:

- Perform an action but do not return a value.

```
public void PMessage() {  
    Console.WriteLine("This is a void function"); }
```

```
class Program
```

```
{
```

```
    static int Add(int a, int b)
```

← Function Signature or Prototype

```
{
```

```
    int sum = a + b;  
    return sum;
```

← Function Definition or Body

```
}
```

```
    static void Main(string[] args)
```

```
{
```

```
    int x, y;  
    x = 10;  
    y = 15;
```

```
    int sum = Add(x, y);
```

← Function Call

```
    Console.WriteLine($"Sum is {sum}");  
    Console.ReadKey();
```

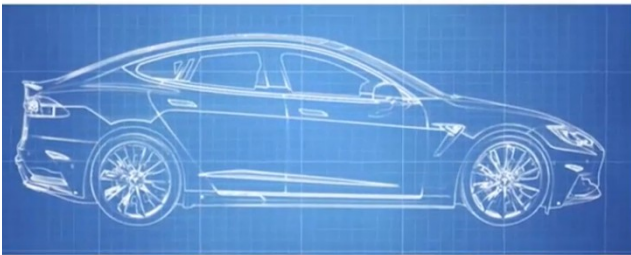
```
}
```

```
}
```

# Object-Oriented Programming (OOP)

- It is a programming paradigm that organizes code into objects, which are instances of classes.
- OOP approach to software development structures a program by **bundling related data (attributes) and behaviors (methods) into a single, cohesive unit.**
- The **central concepts** of OOP are **classes** and **objects**.
  - **Class:** a blueprint or template for creating objects. It defines the structure and behavior that all objects of that type will share.
  - **Object:** a specific instance of a class, created with its own defined data.

Single blueprint(Class)



n number of cars  
(Objects)

Start

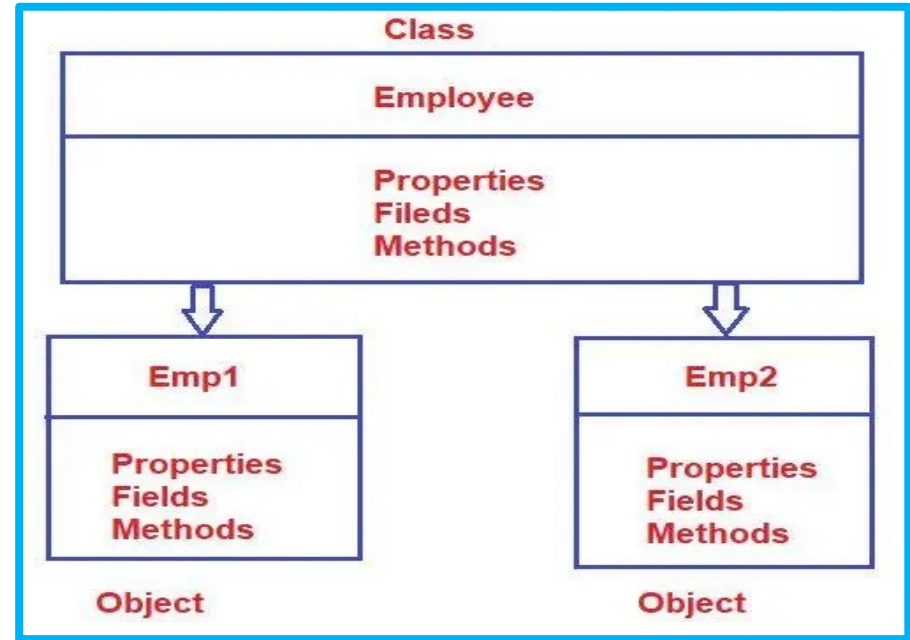
Drive

Stop

Methods

# OOP: Class and Object

- **Class:** A user-defined data type that represents state (attributes/properties) and behavior (methods/functions).
  - Composed of Name, Attributes (fields/properties), Operations (methods/functions)
- **Object:** Can perform activities (methods) and store data (properties).
  - Access class members using the dot (.) operator.
- **Creating a Class and an Object in C#**
  - Class creation: Use the **class** keyword.
  - Object creation: Use the **new** keyword.
  - Access members: `objectName.memberName`.



```
public class Calculator
```

```
{  
    public int CalculateSum(int no1, int no2)  
    {  
        return no1 + no2;  
    }  
}
```

← Class Definition  
Template/blueprint

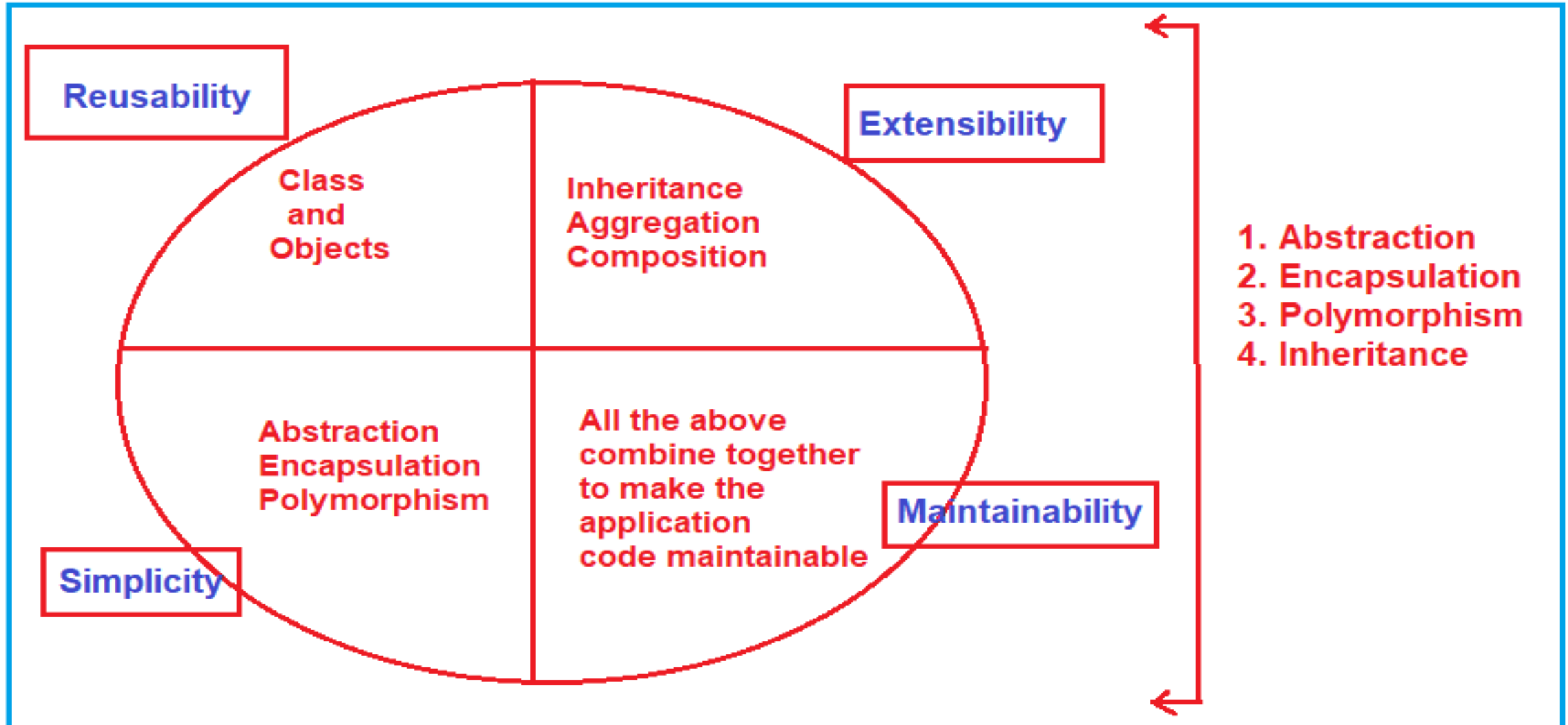
Creating object and accessing class members

```
Calculator calObject = new Calculator();  
calObject.CalculateSum(10, 20);
```

Object

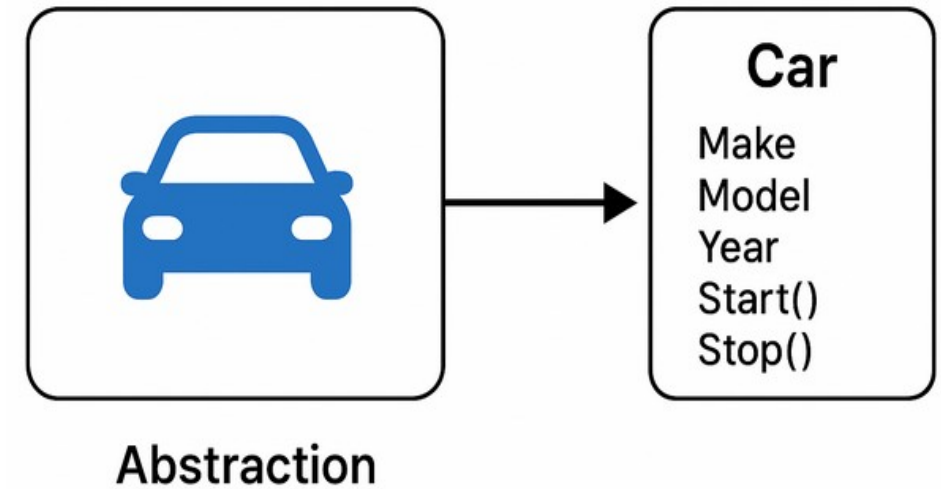
# OOP

- The 4 pillars of OOP are: encapsulation, inheritance, abstraction, and



# OOP: Abstraction

- Abstraction is the process of exposing only essential features of an object while **hiding implementation details**.
  - Focuses on **what an object does**, not how it does it.
  - Helps reduce **complexity** and increase **security**.
  - Supported in C# via **Interfaces** and **Abstract** Classes.
- Real-Time Examples of Abstraction:
  - ATM Machine: You can withdraw money or transfer funds without knowing the internal logic.
  - SMS System: You type a message and send it; the delivery mechanism is hidden.
  - Social Media Login: You enter credentials without knowing the authentication algorithms.





# OOP: Abstraction

- **Abstract Class:** Declared with the **abstract** keyword, it cannot be instantiated directly.
  - It provides a blueprint for derived classes and may contain both abstract (without body) and regular methods.
- **Abstract Methods:** Declared only **within abstract classes**, these methods have no **implementation**.
  - Derived classes must override abstract methods to provide functionality.
- **Interfaces:** Another way to achieve abstraction, enforcing a contract that implementing classes must follow.
- Abstraction ensures cleaner design, reusability, and flexibility, making it highly valuable in event-driven programming where different objects may share common behaviors but handle events differently.

# OOP: Abstraction

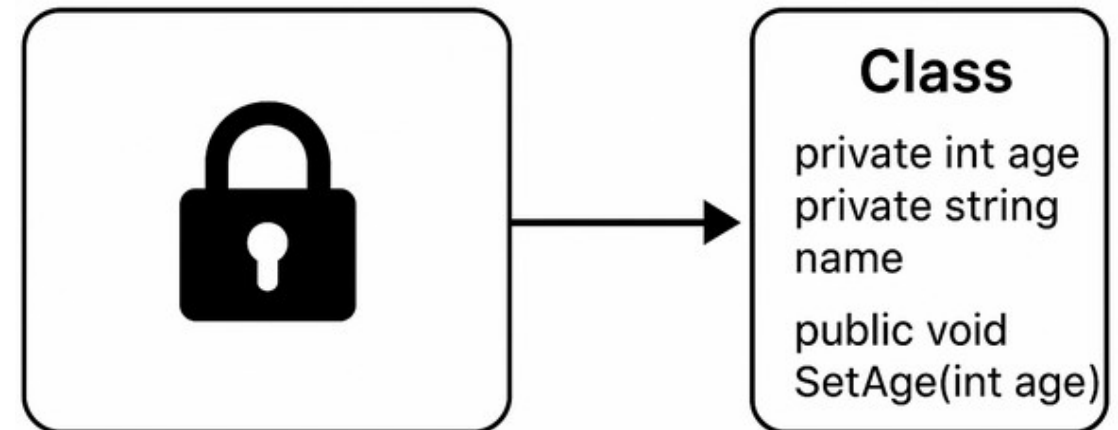
```
abstract class Animal //Abstract Class
{
    public abstract void MakeSound(); // abstract method
}
class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Bark");
    }
}
```

# OOP: Encapsulation

- Encapsulation is the process of **hiding internal data** and mechanics of a class and interacting through controlled, public methods or properties.
- Protects object integrity, prevents unauthorized access, and improves maintainability.
- Use access **modifiers (private, public, protected)** and properties (get/set) to control access.



Example: books, notebooks, pens, pencils, ruler, eraser, and others are stored in a school bag.



**Encapsulation**

# OOP: Encapsulation

```
using System;
class BankAccount
{
    // Private field: hidden from outside classes
    private double balance; // Encapsulated data
    // Public property: controlled access to the private field
    public double Balance
    {
        get
        {
            return balance; // Getter allows reading the private field
        }
        set
        {
            if (value >= 0) // Validation logic, Setter allows controlled modification
                balance = value;
            else
                Console.WriteLine("Invalid balance"); // Protects the field
        }
    }
}
```

```
class Program
{
    static void Main()
    {
        BankAccount account = new BankAccount();
        // Using the property to set the balance
        account.Balance = 500; // Encapsulation in action
        // Using the property to get the balance
        Console.WriteLine(account.Balance); //access controlled
safely
        // Attempting to set an invalid value
        account.Balance = -100; //encapsulation prevents this
    }
}
```

# OOP: Abstraction Vs Encapsulation

- **Encapsulation:** Hides data by bundling variables and methods in a class.
- Use private variables + public getter/setter or properties.
- It is used for Data protection, validation, and security.

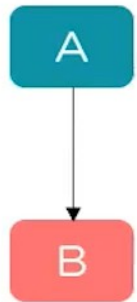
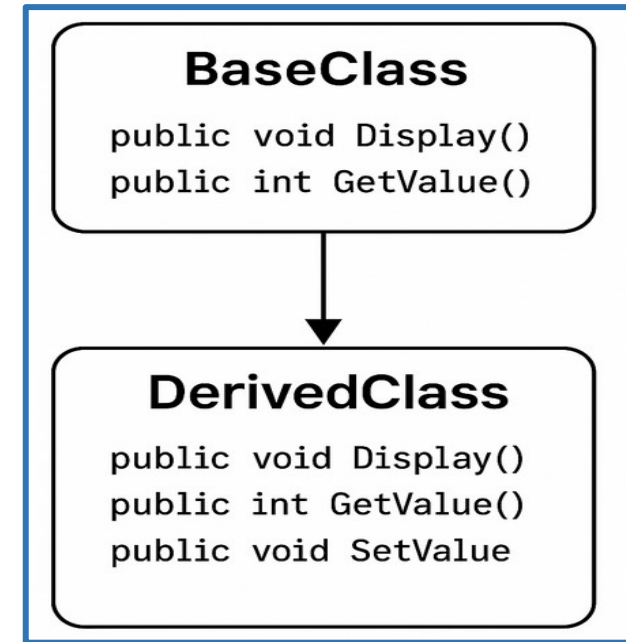
```
private double balance;  
public void SetBalance(double b) { if(b>0) balance=b; }  
public double GetBalance() { return balance; }
```

- **Abstraction:** Hides implementation and exposes only essential methods/services.
- Use interfaces or abstract classes.
- It simplifies usage, reduces complexity, and improves security.

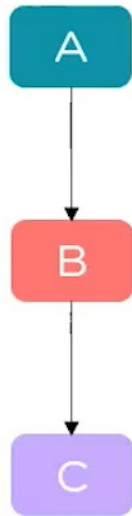
```
public interface IBank { void WithdrawMoney(); }  
class SBI : IBank { public void WithdrawMoney() { /*hidden*/ } }
```

# OOP: Inheritance

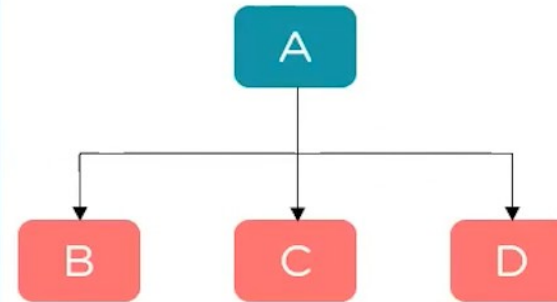
- Inheritance allows a class to acquire fields and methods from another class, promoting code reuse.
- **Base Class(Parent):** The class being inherited from.
- **Derived Class(Child):** The class that inherits from the base class.
- It uses a semicolon(:) to indicate inheritance.



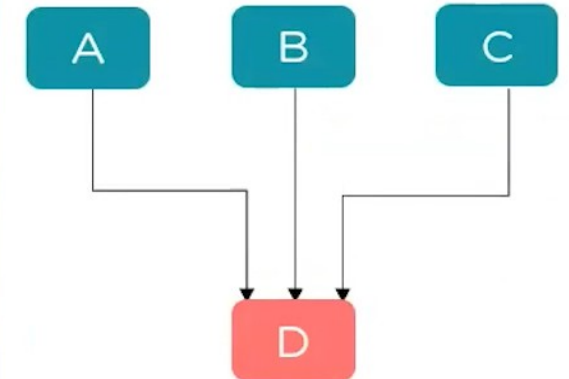
Single Inheritance



Multi-level Inheritance



Hierarchical Inheritance



Multiple Inheritance

# OOP: Inheritance

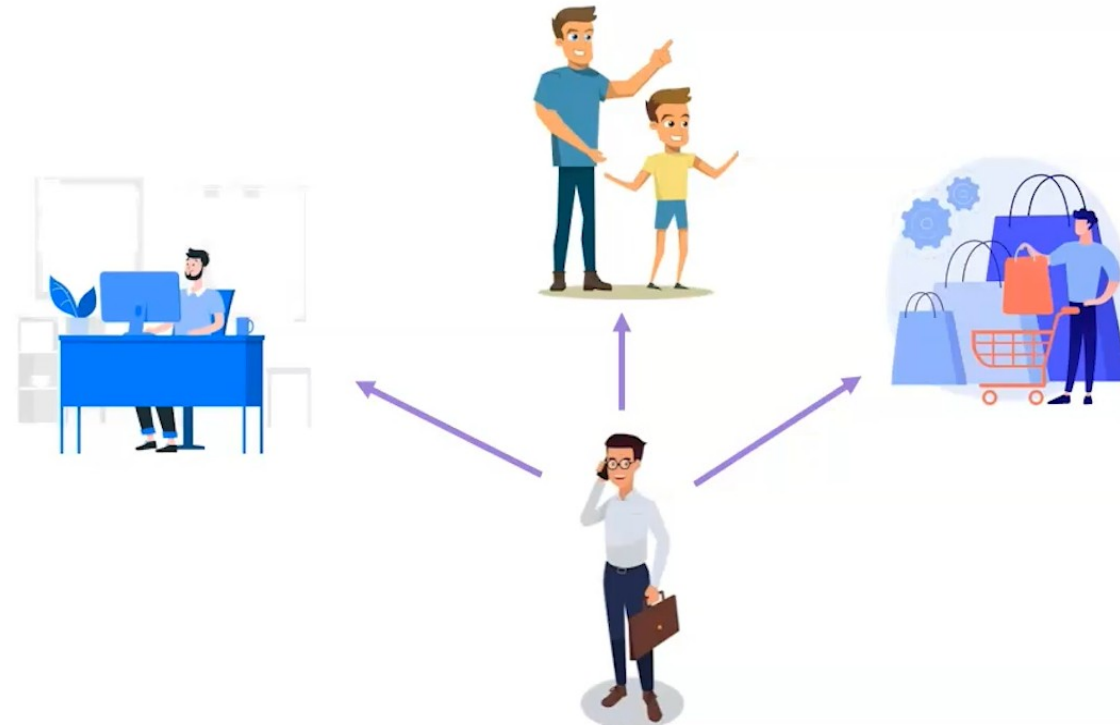
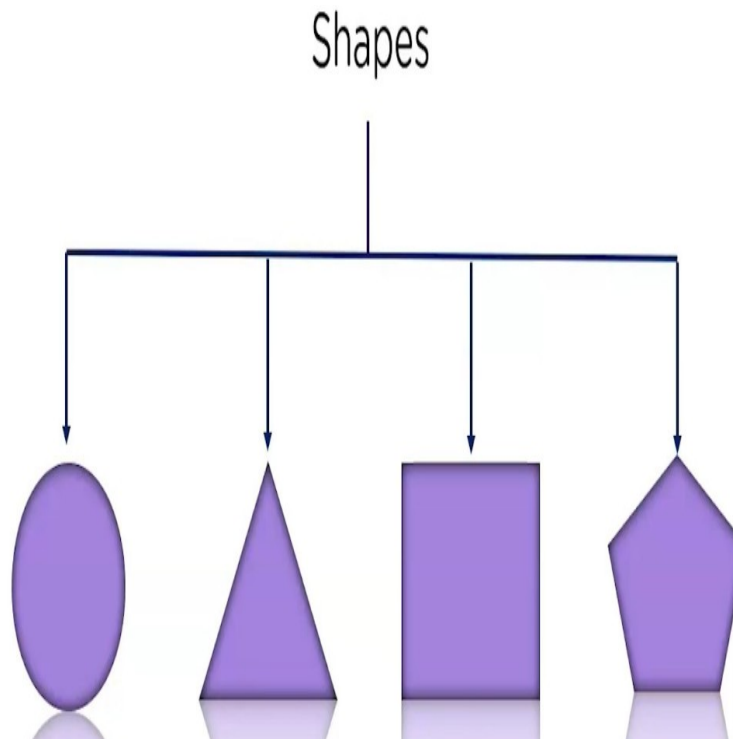
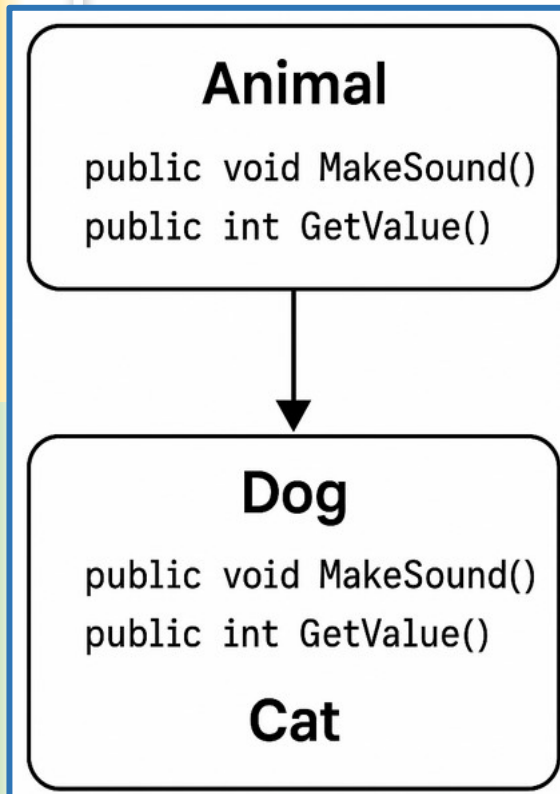
```
class Vehicle //base class (parent) {
    public string brand = "Toyota"; // vehicle field
    public void honk() { // vehicle method
        Console.WriteLine("Toyota Car");
    }
}

class Car : Vehicle //derived class (child) {
    public string modelName = "Toyota2025"; //car field
}

class Program {
    static void Main(string[] args) {
        Car myCar = new Car(); //create a myCar object
        myCar.honk(); // Call the honk() method from the Vehicle class on the myCar object
        Console.WriteLine(myCar.brand + " " + myCar.modelName);
    }
}
```

# OOP: Polymorphism and Overriding Methods

- Polymorphism(many forms) allows objects of different classes to be treated as objects of a common base class.
- It enables a **single method to perform different tasks** based on the object it is acting upon.





# OOP: Polymorphism and Overriding Methods

- **Method Overriding:** A form of polymorphism where a method in a base class is redefined in a derived class using the **override keyword**.
- **The base method must be marked as virtual or abstract to allow overriding.**
  - **Base Class:** The class containing the method to be overridden.
  - **Derived Class:** The class that provides a specific implementation of the method.
  - **virtual Keyword:** Indicates that a method can be overridden in a derived class.
  - **override Keyword:** Used in the derived class to provide a new implementation of the base method.

# OOP: Polymorphism and Overriding Methods

## 1 Static or Compile Time Polymorphism

- Compile-Time/Static Polymorphism (Early Binding):
  - Determined at compile time.
  - Method behavior fixed at compile time.
  - Achieved via Method Overloading or Operator Overloading.

- Example:

```
class Program {  
    public void Add(int a,int b) => Console.WriteLine(a+b);  
    public void Add(float a,float b) => Console.WriteLine(a+b);  
    public void Add(string s1,string s2) => Console.WriteLine(s1+" "+s2);  
}
```

# OOP: Polymorphism and Overriding Methods

2



## Dynamic or Run Time Polymorphism

- **Runtime/Dynamic Polymorphism (Late Binding):**
  - Determined at runtime. Method behavior determined at runtime.
  - Achieved via Method Overriding using **virtual** and **override**.
- Example:

```
class Parent {  
    public virtual void Show() => Console.WriteLine("Parent Show");  
}  
class Child : Parent {  
    public override void Show() => Console.WriteLine("Child Show");  
}  
Parent obj = new Child();  
obj.Show(); // Output Child Show
```

# OOP: Polymorphism

```
class Animal { //base clas
    public virtual void AnimalSound() { //virtual method can
be //overridden in derived classes
        Console.WriteLine("The animal makes a sound");
    } }
class Cat : Animal { //derived class (Child)Cat
    //override base class method to provide specific behavior
    public override void AnimalSound() {
        Console.WriteLine("The cat says meow");
    }
}
class Dog : Animal { //derived class Dog
    //override base class method to provide specific behavior
    public override void AnimalSound() {
        Console.WriteLine("The dog says bow wow");
    } }
```

```
class Program
{
    static void Main(string[] args)
    {
        Animal myAnimal = new Animal(); //base class object
        Animal myCat = new Cat(); //polymorphism: base class
reference to derived object
        Animal myDog = new Dog(); //polymorphism base class
reference to derived object
        myAnimal.AnimalSound(); //calls base class method
        myCat.AnimalSound(); //calls overridden method in Cat
        myDog.AnimalSound(); //calls overridden method in Dog
    }
}
```

# OOP: Constructors

- A constructor is a **special method** used to initialize objects.
  - Name must match the class name.
  - No return type (not even void).
  - Automatically called when an object is created.
  - Can be parameterless or accept parameters to initialize fields.
  - Constructors can be overloaded with multiple signatures.

```
//Parameterless Constructor
class Car{
    public string model;
    public Car() { //constructor, called automatically when an object is created
        model = "Toyota"; //initialize field
    }
    static void Main() {
        Car ford = new Car(); //calls constructor
        Console.WriteLine(ford.model);
    }
}
```

# OOP: Constructors

*//Constructor with Multiple Parameters*

```
class Car{
    public string model;
    public string color;
    public int year;
    //constructor with multiple parameters
    public Car(string modelName, string modelColor, int modelYear) {
        model = modelName;
        color = modelColor;
        year = modelYear;
    }
    static void Main() {
        Car toyota = new Car("Toyota", "White", 1969);
        Console.WriteLine($"{toyota.color} {toyota.year} {toyota.model}");
    }
}
```

# Events

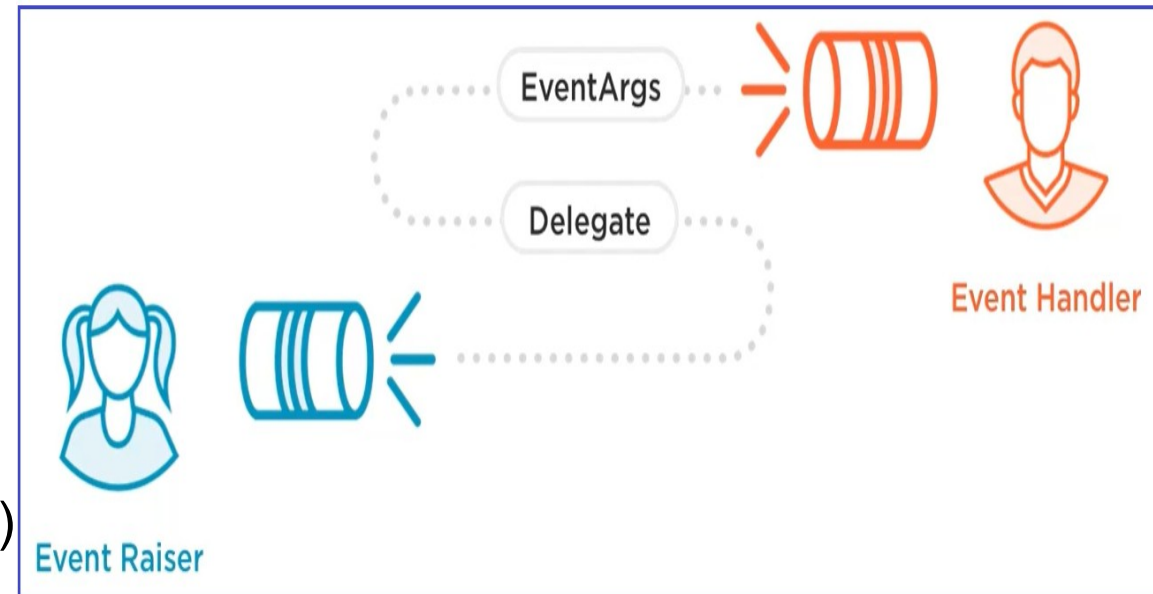
- Events enable a class or object to notify other classes or objects when something of interest occurs.
- Events allow a class or object (the publisher) to notify other classes or objects (subscribers) when something of interest happens.
  - The class that raises the event -> Publisher
  - The class that receives the event -> Subscriber
- Events implement the **Publisher–Subscriber** pattern, providing a structured way for objects to communicate.
- Common usage includes UI actions, like button clicks, in Windows Forms or Web apps, where subscribers respond to user interactions.

# Events

- **Events, Delegates, and Event Handlers** form the foundation of event-driven programming in the .NET Framework.
- They enable communication between objects when an action or change occurs.

- **Real-World Analogy: The Tin Can Example**

- A child speaks into a tin can -> Event Raiser
- The string carries the voice -> Delegate
- The parent listens -> Event Handler
- The message (speech) -> EventArgs (Event Data)
- This analogy illustrates the communication chain:
- Event -> Delegate -> Event Handler (with EventArgs).





# Events

- **Role of Events**
- An event is a notification mechanism that signals when something happens in a program.
- Events act as signals that inform other parts of a system that an action occurred.
- Events allow objects to communicate asynchronously.
- Enables publishers (event raisers) to notify subscribers (event listeners).
- Examples: Button click (Click event), Text change (TextChanged event), Order placed (OrderPlaced event in e-commerce)

```
//Step1: Define one delegate  
public delegate void WorkPerformedHandler(int hours, WorkType workType);
```

```
//Step2: Define one event using the delegate  
public event WorkPerformedHandler WorkPerformed;
```



Events are created using event keyword



Delegate



Event Name

# Events

## ■ Events Characteristics:

- Events are central to the .NET Framework, used across UI, services, and business logic.
- One event can notify multiple subscribers.
- Events often carry additional information via EventArgs.
- Visual Studio icons like the “lightning bolt” ⚡ represent events in the properties window.
- For example, just like an announcement over a loudspeaker alerts many people, an event in C# notifies all registered listeners.

# Events

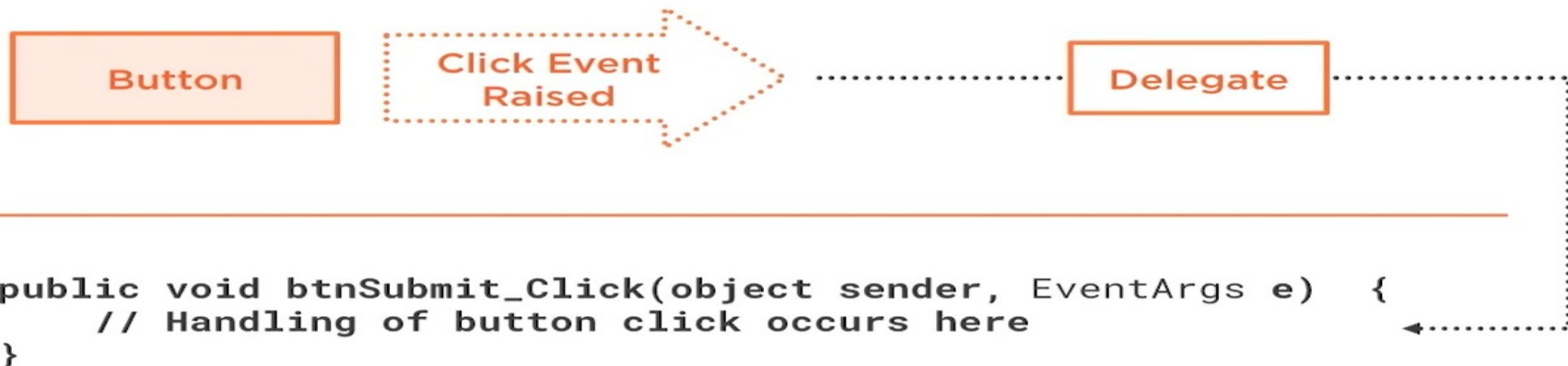
- **Role of Event Handlers**

- An Event Handler is a method that receives and processes the data sent by a delegate when an event occurs.

`void Handler(object sender, EventArgs e)`

- sender: Object that raised the event.
- EventArgs e: Data passed with the event.
- Receive notifications from delegates.
- Access and interpret EventArgs data.
- Execute logic, e.g., update UI, save to database, process order.

An event handler is responsible for accessing data passed by a delegate:



# Delegates

- Delegates an object that knows how to call a method(a group of methods).
- A delegate is a type-safe function pointer in C#.
- It holds a reference to a method/function and allows you to invoke that method indirectly.
- Delegates provide flexibility by allowing methods to be passed as parameters, invoked dynamically, and used for callbacks, events, or asynchronous programming.
- Delegates enable methods to be passed as parameters, especially useful for implementing callback functions, defining abstract algorithms, and event handling that can be customized with specific methods.
- It provides a level of indirection, decoupling the code that defines a task from the code that executes it.

# Delegates

- **Creating a Delegate:** The signature of the delegate must match the method it points to.
  - Delegates can be static or instance methods.
- **Using a Delegate**
  - Declare a delegate
  - Instantiate it with a method
  - Invoke the delegate

```
//Delegate
public delegate void WorkPerformedHandler(int hours, WorkType workType);

//Handler Method
public void Manager_WorkPerformed(int workHours, WorkType wType)
{
    //Processing the Data
}
```

## How to use the Delegate in C#?

### Step1: Declaring the Delegate Signature

```
public delegate void WorkPerformedHandler(int hours, WorkType workType);
```

### Step2: Creating the Handler Method with the same signature as Delegate

```
public static void Manager_WorkPerformed(int workHours, WorkType wType)
{
    //Processing the Data
}
```

### Step3: Creating an Instance of Delegate

```
WorkPerformedHandler del1 =
    new WorkPerformedHandler(Manager_WorkPerformed);
```

# Delegates

- How to invoke a delegate in C#?

```
public delegate void WorkPerformedHandler(int hours, WorkType workType);

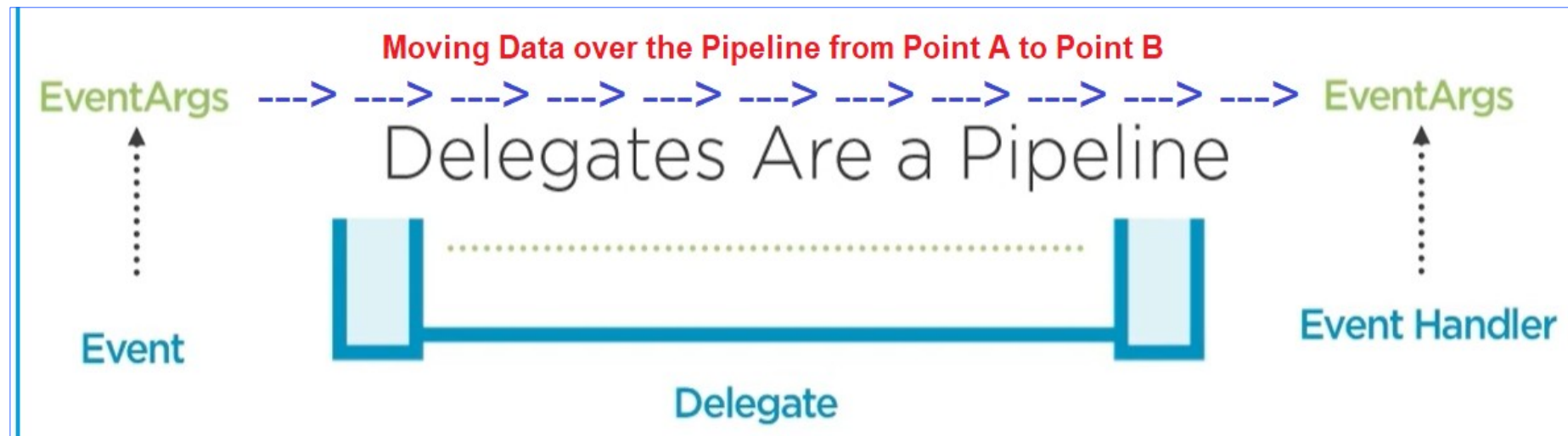
WorkPerformedHandler del1 =
    new WorkPerformedHandler(Manager_WorkPerformed);
del1(10, WorkType.Golf);
```

```
public static void Manager_WorkPerformed(int workHours, WorkType wType)
{
    //Processing the Data
}
```

Invoking a delegate: calling (executing) the method(s) that the delegate points to.

# Delegates

- **Role of Delegates**
- A type-safe function pointer that defines the signature of methods responding to events.
- Acts as a communication channel between the Event and its Handler.
- Delegates are the pipelines that connect events to handlers.
- Enables passing of data (EventArgs) from Event Raiser -> Event Handler.
- Allows multiple listeners to subscribe to the same event.
- All delegates in .NET inherit from MulticastDelegate.
- Delegates are the pipelines that connect events to handlers.



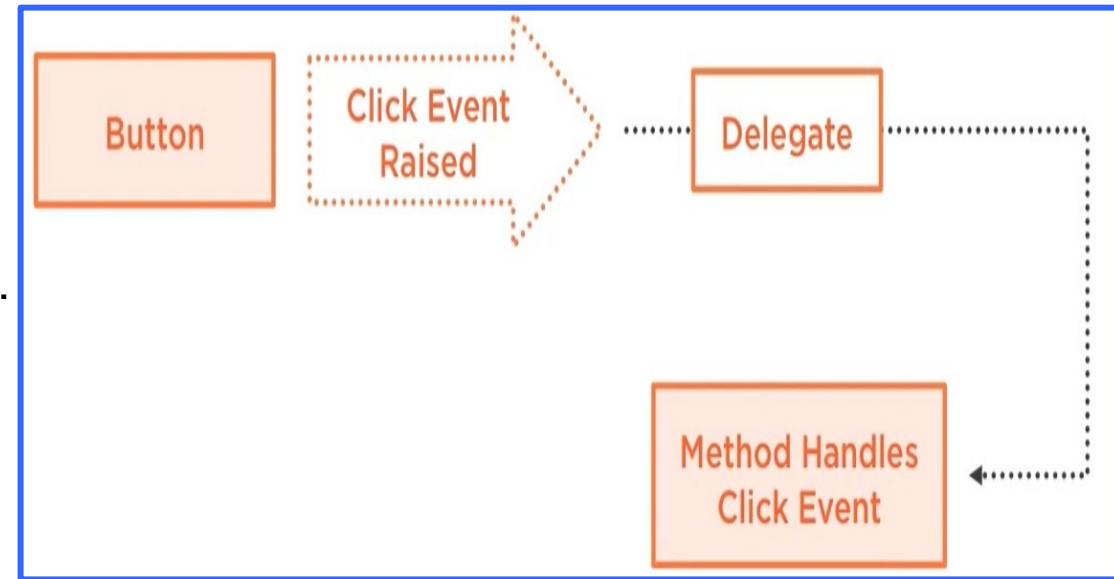
# Delegates

## ■ Types of Delegates

- **Single-cast Delegate:** Points to one method.
- **Multicast Delegate:** Points to multiple methods.
  - Delegates can hold references to multiple methods.
  - Use += to add and -= to remove methods. ‘
  - All methods are called sequentially when invoked.

## ■ Delegate as a Function Pointer

- Delegates point to methods (event handlers).
- When an event is raised, the delegate calls the method it references.
- This is why delegates are known as function pointers in C#.
- Example: when a button's Click event fires, the delegate passes event data to the associated method (e.g., btnSubmit\_Click), which executes the response code.





# Delegates vs Events

- **Delegate:** A type-safe function pointer that can hold references to one or more methods with a specific signature and return type.
  - Delegates define the signature of methods that can be called.
  - Delegates act as the communication channel between the publisher and subscribers.
- **Event:** A wrapper around a delegate that enforces encapsulation, allowing other classes to subscribe or unsubscribe, but not invoke the delegate directly.
  - Events use delegates to allow other classes to subscribe (listen) and respond when something happens.
  - An event is essentially a wrapper around a delegate; it adds an extra layer of control and safety.
  - Events restrict direct delegate invocation from outside the publishing class.
  - Provide a standardized and secure mechanism for notifications in .NET.
  - Support multiple subscribers for one event.

# Events vs Delegates

```
using System;
public class BankAccount {
    //Step 1 Declare a delegate type
    public delegate void BalanceChangedHandler(decimal newBalance);
    //Step 2 Declare an event based on that delegate
    public event BalanceChangedHandler BalanceChanged;
    private decimal balance;
    public void Deposit(decimal amount) {
        balance += amount;
        Console.WriteLine($"Deposited: {amount}");
        OnBalanceChanged();
    }
    public void Withdraw(decimal amount) {
        if (amount <= balance) {
            balance -= amount;
            Console.WriteLine($"Withdrawn: {amount}");
            OnBalanceChanged();
        }
        else{
            Console.WriteLine("Insufficient funds!");
        }
    }
    //Step 3 Protected method to raise the event
    protected virtual void OnBalanceChanged() {
        BalanceChanged?.Invoke(balance);
    }
}
//Step 4 Subscriber class
public class SMSNotifier {
    public void SendSms(decimal newBalance) {
        Console.WriteLine($"[SMS] Your new balance is {newBalance:C}");
    }
}
```

```
public class Program {
    public static void Main() {
        BankAccount account = new BankAccount();
        SMSNotifier notifier = new
        SMSNotifier();
        //Step 5 Subscribe to event
        account.BalanceChanged +=
        notifier.SendSms;

        account.Deposit(1000);
        account.Withdraw(300);
    }
}
```

# Namespace

- A namespace is a **logical container** for classes, interfaces, methods, structs, enums, and other namespaces.
- It is used to organize code & avoid naming conflicts in large programs or projects.
- In C#, a **namespace** is a logical, organizational container, while a **library** is a physical container (a compiled file) that holds reusable code.
- A single library can contain **multiple** namespaces, and a **single** namespace can **span multiple libraries**. It can contain **nested** namespaces.
- Namespace: prevents naming conflicts, organizes code, access to members

```
using System; //importing System namespace (built-in namespace)
namespace MyApp { //a custom namespace for your project
    class Program {
        static void Main() {
            Console.WriteLine("Hello, World!"); //using System allows us to use Console directly
        } } }
```

# Library

- Library is a collection of precompiled code (classes, methods, etc.) that can be reused in programs.
- A class library is a project that packages reusable code, which is compiled into a **Dynamic Link Library (.dll)** file.
- Other projects can then reference this .dll to use the classes and methods it contains.
- Saves time and avoids rewriting common functionality.
- **Library Types in C#:**
  - Class Library (DLL): .dll files that can be referenced in projects.
  - **.NET Framework/.NET Core Libraries:** Prebuilt libraries like System, System.IO, System.Collections.Generic.
- Developers can also create custom libraries (DLLs) for reuse in multiple projects.

```
using System.IO; // using File and Directory classes from System.IO library
class Program {
    static void Main() {
        File.WriteAllText("example.txt", "Hello Library!");
    }
}
```

**Thank You!**

**?**